



Trabalho Prático

Grupo C

Resolução de Problemas com Grafos

INTEGRANTES

-  Igor Praciano Thomaz
-  Rafael Lima Cacau
-  Tiago Silveira Taumaturgo

Unidade 3 - T1 | Problema: Treehouses

O Problema Treehouses (Kattis)

? O que precisamos resolver?

- Conectar todas as casas na árvore com o menor comprimento total de cabos.
- Aproveitar as conexões que já existem sem custo extra.

i Contexto do Desafio

- ✓ Algumas casas já estão conectadas à rede principal.
- ✓ Pares específicos já possuem cabos instalados.

Objetivo Final

Calcular o **custo adicional mínimo** necessário para a conectividade total.

Modelagem como Grafo

Componentes do Grafo



Vértices

Cada Treehouse é um nó no grafo



Arestas

Possíveis conexões entre as casas



Pesos

Distância física entre as coordenadas

DISTÂNCIA EUCLIDIANA

$$\text{dist}(i, j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$$

Grafo não direcionado, ponderado e completo (K_n)

A Variação do MST

Conexões Gratuitas

- ✓ **Rede Inicial:** As primeiras e treehouses já estão interconectadas.
- ✓ **Cabos Existentes:** p pares informados já possuem cabos.

Essas conexões entram no grafo com:

PESO 0.0

O Diferencial

Não buscamos o custo total de conectar tudo do zero.

A resposta é o **menor comprimento adicional** necessário via algoritmo de Prim.

Estratégia Algorítmica

1

Inicialização de Conexões Gratuitas

Adicionar arestas de **peso zero** para as primeiras e casas e os p pares existentes.

2

Construção do Grafo Completo

Calcular as distâncias euclidianas para todos os outros pares de treehouses possíveis.

3

PrimMST (Eager)

Crescer a MST a partir de um vértice, mantendo *distTo* e *edgeTo* via **Fila de Prioridade Indexada**.

4

Resultado Final

A soma dos pesos das arestas escolhidas resulta no **menor custo adicional** de cabos.

Implementação em Java - Parte 1

```
7 public class Main {  
8  
9     Run | Debug  
10    public static void main(String[] args) throws Exception {  
11        java.io.File file = new java.io.File("dados/dados02.txt");  
12        if (file.exists()) {  
13            System.setIn(new java.io.FileInputStream(file));  
14        }  
15  
16        int n = StdIn.readInt();  
17        int e = StdIn.readInt();  
18        int p = StdIn.readInt();  
19  
20        double[] x = new double[n];  
21        double[] y = new double[n];  
22  
23        for (int i = 0; i < n; i++) {  
24            x[i] = StdIn.readDouble();  
25            y[i] = StdIn.readDouble();  
26        }  
27  
28        EdgeWeightedGraph graph = new EdgeWeightedGraph(n);
```

Leitura das coordenadas e inicialização do grafo

Implementação em Java - Parte 2

```
30
31     for (int i = 1; i < e; i++) {
32         graph.addEdge(new Edge(v: 0, i, weight: 0.0));
33     }
34
35     for (int i = 0; i < p; i++) {
36         int a = StdIn.readInt() - 1;
37         int b = StdIn.readInt() - 1;
38         graph.addEdge(new Edge(a, b, weight: 0.0));
39     }
40
41     for (int i = 0; i < n; i++) {
42         for (int j = i + 1; j < n; j++) {
43             double dx = x[i] - x[j];
44             double dy = y[i] - y[j];
45             double dist = Math.sqrt(dx * dx + dy * dy);
46             graph.addEdge(new Edge(i, j, dist));
47         }
48     }
49
50     PrimMST mst = new PrimMST(graph);
51
52     System.out.printf(Locale.US, "%.10f\n", mst.weight());
53 }
```

Instanciação do PrimMST e formatação da saída (10 casas decimais)

Complexidade da Solução

Etapa do Algoritmo	Complexidade	Descrição
Geração de Arestas	$O(v^2)$	Cálculo de distâncias para o grafo completo.
PrimMST (Eager)	$O(e \log v)$	Onde $e \approx v^2/2$ arestas totais no grafo completo.

COMPLEXIDADE TEMPORAL TOTAL

$O(v^2 \log v)$

COMPLEXIDADE ESPACIAL

$O(v^2)$

Casos Especiais e Implementação

⚙️ Detalhes Técnicos

- **Conversão de Índices:** A entrada do Kattis usa base 1, convertida para base 0 no Java.
- **Precisão Decimal:** Saída obrigatória com 10 casas decimais para aceitação.

```
System.out.printf("%.10f\n", totalCost);
```

⚠️ Casos de Borda

- **Conectividade Inicial:** Se as conexões gratuitas já unem todas as casas, o custo é 0.0.
- **Grafo Completo:** Tratamento de $N(N-1)/2$ arestas para garantir a otimalidade.
- **Linguagem:** Uso de Fila de Prioridade Indexada para eficiência no Prim.

ACCEPTED

Conclusão e Resultados

Problema Treehouses resolvido com PrimMST.

- 💡 Modelar conexões existentes como arestas de peso zero simplifica a construção da MST.
- ✅ O algoritmo de Prim com Fila de Prioridade Indexada garante a escolha eficiente das conexões.
- 📈 A complexidade $O(v^2 \log v)$ é ideal para o grafo completo do problema.